

Nome: Eduardo Lucas Lemes Januário CT3037568

Nome: Guilherme Batista CT3037274

Teórico - Algoritmos de Ordenação

1) Explique, com suas palavras, as principais diferenças entre algoritmos de ordenação simples (Seleção, Inserção, Bolha) e eficientes (MergeSort, QuickSort, CountingSort, RadixSort).

Algoritmos de ordenação simples (Seleção, Inserção, Bolha):

Ideia geral: usam comparações diretas entre pares de elementos, de forma repetitiva. Complexidade: normalmente $O(n^2)$ no pior e caso médio, o que os torna pouco eficientes em vetores grandes.

Características:

- Fáceis de implementar, bons para aprendizado.
- Funcionam bem apenas para pequenos conjuntos de dados.
- Inserção pode ser eficiente em listas quase ordenadas.
- Seleção faz o mesmo número de comparações, mesmo que os dados já estejam quase prontos.
- Bolha é o mais ineficiente, mas fácil de entender.

Algoritmos de ordenação eficientes (MergeSort, QuickSort, CountingSort, RadixSort):

Ideia geral: usam estratégias mais avançadas (divisão e conquista, contagem, distribuição). Complexidade: em geral $O(n \log n)$ ou até $O(n)$ (para algoritmos não baseados em comparação, como Counting e Radix).

Características:

- Escalam bem para grandes volumes de dados.
- MergeSort: estável e garante $O(n \log n)$, mas consome memória extra.
- QuickSort: em média o mais rápido em prática, mas pode ter pior caso $O(n^2)$ sem boas escolhas de pivô.
- CountingSort: ideal para chaves inteiras em intervalo pequeno, com complexidade linear.
- RadixSort: eficiente para ordenar números de muitos dígitos (ou strings), baseando-se em posições (radix).

2) Associe cada método de ordenação abaixo à situação em que seria mais adequado:

(Inserção) Ordenar um número pequeno de elementos quase ordenados.

(QuickSort) Ordenar uma grande quantidade de números sem repetição.

(CountingSort) Ordenar registros que possuem chaves inteiras pequenas e limitadas.

(RadixSort) Ordenar números com muitos dígitos, sem comparações diretas.

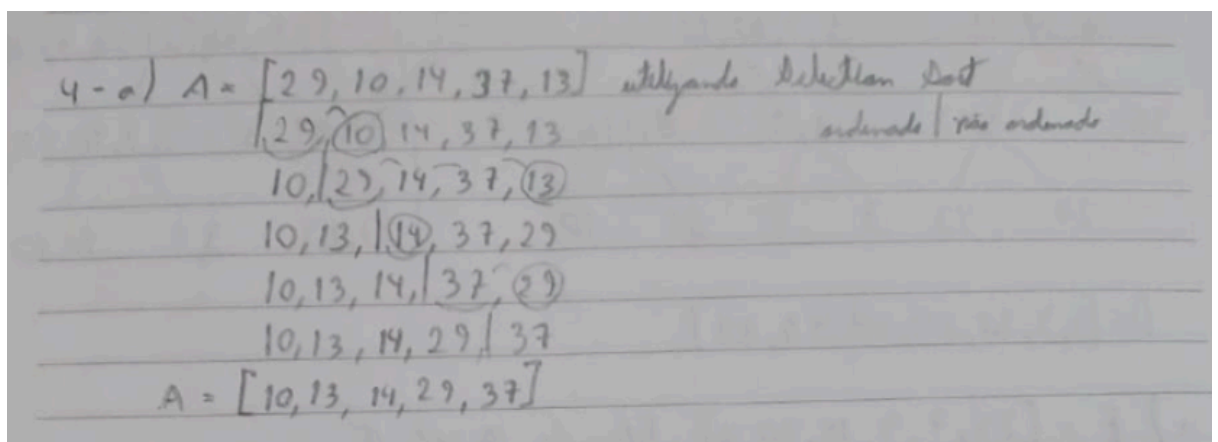
(Seleção) Ordenar um vetor de tamanho médio, sem informações extras sobre os dados.

3) Construa uma tabela apresentando as complexidades de pior caso, caso médio e de espaço dos métodos de ordenação estudados.

Algoritmo	Melhor Caso	Caso Médio	Pior Caso	Espaço Extra
Seleção	$O(n^2)O(n^2)O(n^2)$	$O(n^2)O(n^2)O(n^2)$	$O(n^2)O(n^2)O(n^2)$	$O(1)O(1)O(1)$
Inserção	$O(n)O(n)O(n)$	$O(n^2)O(n^2)O(n^2)$	$O(n^2)O(n^2)O(n^2)$	$O(1)O(1)O(1)$
Bolha	$O(n)O(n)O(n)^*$	$O(n^2)O(n^2)O(n^2)$	$O(n^2)O(n^2)O(n^2)$	$O(1)O(1)O(1)$
MergeSort	$O(n \log n)O(n \log n)O(n \log n)$	$O(n \log n)O(n \log n)O(n \log n)$	$O(n \log n)O(n \log n)O(n \log n)$	$O(n)O(n)O(n)$
QuickSort	$O(n \log n)O(n \log n)O(n \log n)$	$O(n \log n)O(n \log n)O(n \log n)$	$O(n^2)O(n^2)O(n^2)$	$O(\log n)O(\log n)O(\log n)$ (recursão)
CountingSort	$O(n+k)O(n+k)O(n+k)$	$O(n+k)O(n+k)O(n+k)$	$O(n+k)O(n+k)O(n+k)$	$O(n+k)O(n+k)O(n+k)$
RadixSort	$O(nk)O(nk)O(nk)$	$O(nk)O(nk)O(nk)$	$O(nk)O(nk)O(nk)$	$O(n+k)O(n+k)O(n+k)$

4) Faça o que se pede para cada vetor

a) Ordene o vetor $A = [29, 10, 14, 37, 13]$ utilizando Selection Sort. Mostre o vetor após cada iteração da escolha do menor elemento.



b) Ordene o vetor $B = [8, 3, 5, 4, 6]$ utilizando Insertion Sort. Mostre o vetor a cada inserção.

b) $B = [8, 3, 5, 4, 6]$ utilizando Insertion Sort

	ordenado	não ordenado
8, 3, 5, 4, 6	8	3, 5, 4, 6
3, 8, 5, 4, 6	3, 8	5, 4, 6
3, 5, 8, 4, 6	3, 5, 8	4, 6
3, 4, 5, 8, 6	3, 4, 5, 8	6
3, 4, 5, 6, 8	3, 4, 5, 6, 8	
$B = [3, 4, 5, 6, 8]$		

c) Ordene o vetor $C = [20, 35, 15, 40, 10]$ utilizando Bubble Sort. Mostre o vetor a cada passagem.

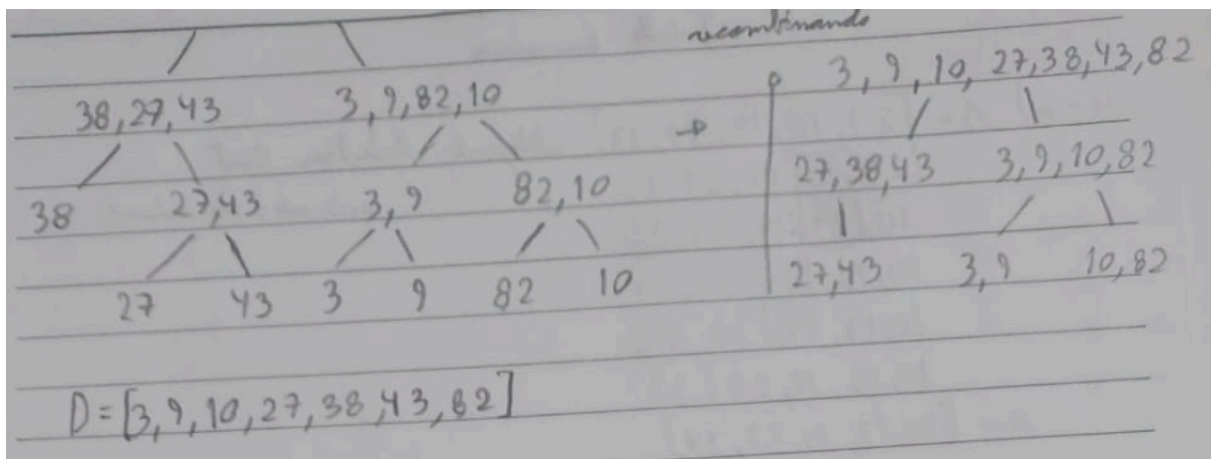
c) $C = [20, 35, 15, 40, 10]$ utilizando Bubble Sort

	não ordenado	ordenado
20, 35, 15, 40, 10	20, 35, 15, 40, 10	
20, 15, 35, 40, 10	20, 15, 35, 40, 10	
20, 15, 35, 10, 40	20, 15, 35, 10, 40	40
15, 20, 35, 10, 40	15, 20, 35, 10, 40	40
15, 20, 10, 35, 40	15, 20, 10, 35, 40	40
15, 10, 20, 35, 40	15, 10, 20, 35, 40	40
10, 15, 20, 35, 40	10, 15, 20, 35, 40	40
$C = [10, 15, 20, 35, 40]$		

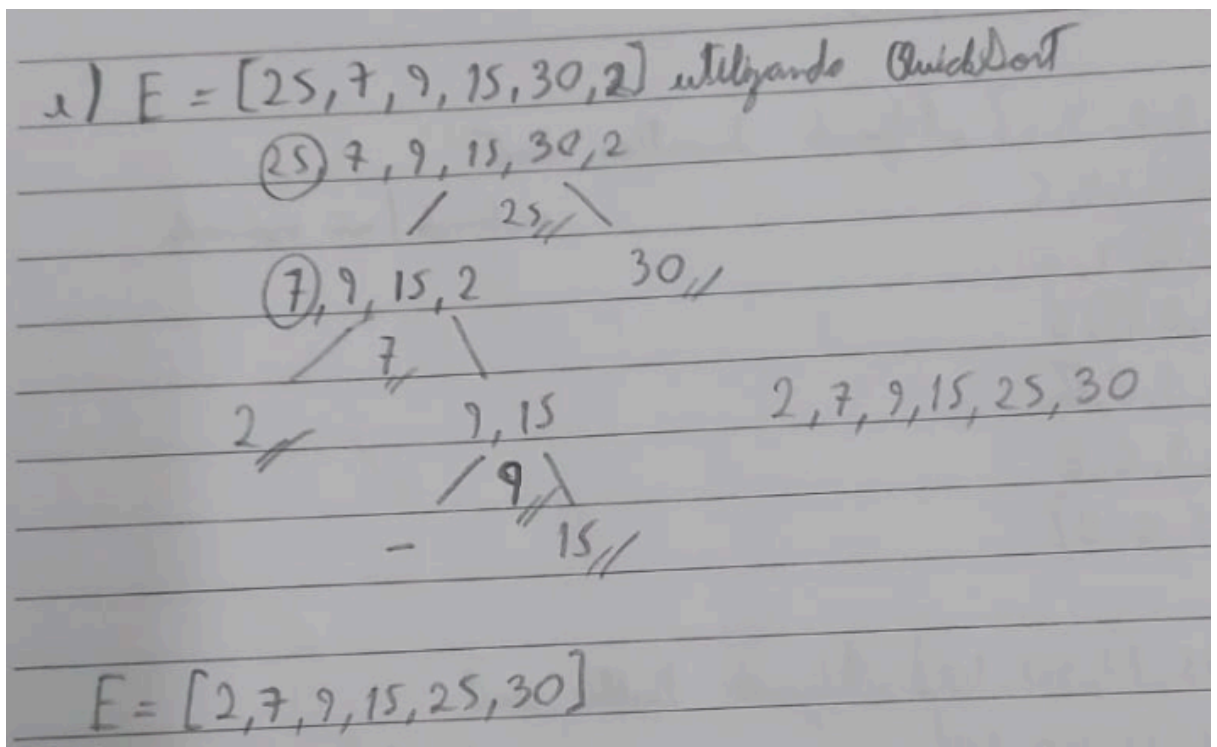
d) Mostre a execução do MergeSort para o vetor $D = [38, 27, 43, 3, 9, 82, 10]$, destacando como o vetor é dividido e depois intercalado.

d) $D = [38, 27, 43, 9, 9, 82, 10]$ utilizzando e dimostrando MergeSort

38, 27, 43, | 9, 9, 82, 10



e) Ordene o vetor $E = [25, 7, 9, 15, 30, 2]$ utilizando QuickSort com o primeiro elemento como pivô. Mostre os passos da partição.



f) Ordene o vetor $F = [4, 2, 2, 8, 3, 3, 1]$ utilizando Counting Sort. Mostre o vetor de contagem e a reconstrução do vetor final.

p) $F = [4, 2, 2, 8, 3, 3, 1]$ utilizando Counting Sort
 $4, 2, 2, 8, 3, 3, 1 \rightarrow C = [0 \dots 8]$
 $C = 0, 1, 2, 2, 1, 0, 0, 0, 1$
 $ 0 1 2 3 4 5 6 7 8$
 $F = 1, 2, 2, 3, 3, 4, 8$

g) Ordene o vetor $G = [329, 457, 657, 839, 436, 720, 355]$ utilizando Radix Sort. Mostre a ordenação dígito a dígito.

g) $G = [329, 457, 657, 839, 436, 720, 355]$ utilizando Radix Sort
 $329, 457, 657, 839, 436, 720, 355$
 unidades |
 720, 355, 436, 457, 657, 329, 839
 dezenas |
 720, 329, 436, 839, 355, 457, 657
 centenas |
 $G = [329, 355, 436, 457, 657, 720, 839]$

5) Qual algoritmo você considera mais fácil de executar manualmente? E qual seria o mais eficiente para grandes conjuntos de dados? Justifique.

O Insertion Sort é o mais fácil de executar manualmente, pois se percorre e insere cada elemento no lugar certo, logo só é preciso comparação e deslocamento. O Radix Sort é o mais eficiente para grandes conjuntos de dados porque é quase linear, já que é explorado a estrutura dos números.

6) Considere o vetor A escreva o passo a passo para encontrar as chaves abaixo utilizando a busca proporcional:

$$A = [5, 12, 20, 28, 35, 42, 50, 58, 65, 73, 80, 88, 95, 102, 110]$$

a) 35

b) 88

c) 60

Handwritten notes showing the steps of a proportional search algorithm:

6) $A = [5, 12, 20, 28, 35, 42, 50, 58, 65, 73, 80, 88, 95, 102, 110]$
Utilizando busca proporcional $\rightarrow$ base = $\frac{110 - 5}{14} = 7,5$

a) 35 $\frac{35 - 5}{7,5} = 4$ 35 $\rightarrow$ posição 4 //
no vetor $\uparrow$

b) 88 $\frac{88 - 5}{7,5} = 11$ 88 $\rightarrow$ posição 11 //

c) 60 $\frac{60 - 5}{7,5} = 7,333...$ 60 $\rightarrow$ não encontrado //

$\hookrightarrow$ entre 58 e 65